Encoding call-by-push-value in the π-calculus

Benjamin Bennetzen
Department of Computer Science
Aalborg University
Aalborg, Denmark
bcb@cs.aau.dk

Hans Hüttel
Department of Computer Science
Aalborg University
Aalborg, Denmark
hans@cs.aau.dk

Nikolaj Rossander Kristensen
Department of Computer Science
Aalborg University
Aalborg, Denmark
nrk@cs.aau.dk

Peter Buus Steffensen
Department of Computer Science
Aalborg University
Aalborg, Denmark
psteff19@student.aau.dk

We define an encoding of a reduced call-by-push-value λ-calculus (CBPV) into the internal π-calculus. The encoding uses the restricted output discipline of the internal pi-calculus together with explicit handles for application and return, allowing CBPV substitution to be represented by name passing and replicated pointer processes. We prove an operational correspondence theorem for the encoding: every CBPV reduction is simulated by internal reductions of the encoded process, and every reduction sequence of an encoded process corresponds, up to weak bisimilarity, to a CBPV reduction. We also adapt Gorla's criteria for good encodings to the present setting and show that the encoding satisfies compositionality, name invariance, operational correspondence, divergence reflection, and success sensitiveness. Finally, we compare the encoding with Milner's classical encodings of call-by-value and call-by-name λ-calculi into the π-calculus. Composing Levy's translations of call-by-value and call-by-name into CBPV with our encoding yields processes that are behaviourally close to Milner's direct encodings, while making explicit the additional thunk and return structure introduced by CBPV.

1 Introduction

The notions of call-by-value and call-by-name as parameter-passing mechanisms date back to the setting of ALGOL60 [2]. Later, Plotkin [16] gave a detailed account of these two evaluation strategies in the setting of the untyped λ-calculus.

Levy showed [12, 13] how to define the notion of call-by-push-value that subsumes both call-by-value and call-by-name in an enriched version of the λ-calculus that makes use of the well-established notion of thunks, which was first used for implementing call-by-name in ALGOL60, as well as a variation of the return-construct known from monads that can be used to describe how values and terms are passed in a computation.

There has also been interest in studying how to reason about computations that use call-by-push-value. Rizkallah et al. [17] have presented an equational theory, which is essentially the congruence closure of the reduction relation and formalized it in Rocq.

In the setting of process calculi, there has also been a line of work that studies encodings of the λ-calculus into a given process calculus. This began with the seminal work by Milner [14] who showed that it is straightforward to encode both call-by-value and call-by-name reduction strategies into the π-calculus. Since then, there has been work on understanding the expressive

Submitted to:
Express/SOS 2025

power of process calculi by using encodings of this kind as a yardstick. Sangiorgi has shown [19] that the use of a higher-order π-calculus makes the description of the encoding simpler, and later work has shown that both the asynchronous π-calculus [25] and the "internal" π-calculus πI [21] known as the πI-calculus allow for encodings of the λ-calculus. In [8], Durier et al. study which equivalence on λ-terms is induced by Milner's encodings, and to this end they make use of the πI-calculus as well as the asynchronous π-calculus.

In the realm of formalizations, Accattoli et al. have given a formal proof of the correctness of Milner's original encodings [1] using the Abella proof assistant.

This paper makes the following contributions: First, we define an encoding of call-by-push-value λ-calculus (CBPV) in the internal π-calculus (πI-calculus), a restricted variant of the π-calculus in which outputs are bound. Second, we prove the central operational correspondence result for this encoding, namely soundness and completeness with respect to CBPV reduction. Third, we show that the encoding satisfies Gorla's criteria for good encodings, suitably adapted to the present setting. Fourth, we informally compare the resulting CBPV to πI-calculus encoding with Milner's direct encodings of call-by-value and call-by-name, showing how the variable and abstraction cases align and where the CBPV translation changes the shape of the processes.

This paper is a condensed version of a more comprehensive full paper that includes complete proofs, a preliminary Rocq formalization of the central soundness argument, and variations of the encoding using the monadic π-calculus, asynchronous π-calculus and local π-calculus [3].

The choice of the πI-calculus originally arose from the observation that only bound outputs are necessary in an encoding.

However, there are other important benefits of a fundamental nature that come from this choice. Firstly, the concern of which notion of bisimulation equivalence to consider (late, early or open) and how to deal with congruence properties are easily resolved: In the πI-calculus, the notions of late, early and open bisimilarity coincide and are congruence relations.

Secondly, this choice has also made it easier to formalize the encoding in Rocq, as it simplifies the treatment of α-conversion in the π-calculus.

Throughout this paper, we use the polyadic variant of the πI-calculus, where outputs can send tuples of names. This is a straightforward generalization of the monadic varition, where processes are allowed to send and receive tuples of names. This is done out of convenience to allow for more compact encodings, as it avoids the need for intermediate channels to sequentially transmit multiple names. When we refer to the πI-calculus in this paper, we mean the polyadic variant unless otherwise specified.

The paper is structured as follows. In section 2, we recall the call-by-value and call-by-name variants of the λ-calculus. In section 3, we introduce the call-by-push-value λ-calculus and its encodings of call-by-value and call-by-name. In section 4, we present the internal π-calculus and explain why it is a suitable target calculus for our encoding. In section 5, we define the encoding of CBPV in the internal π-calculus and prove the central soundness and completeness results. In section 6, we discuss the encoding with respect to Gorla's criteria for good encodings, and in section 7, we compare it with Milner's original encodings of call-by-value and call-by-name. Finally, section 8 concludes and outlines directions for future work.

2 Call-by-value and call-by-name

In this section we will give an introduction to the call-by-value (CBV) and call-by-name (CBN) paradigms of the λ-calculus. This serves as motivation for why one would want to use a calculus that encompasses both of these paradigms.

We first present the notions of call-by-value (CBV) and call-by-name (CBN), as they appear in the λ-calculus.

The syntax and semantics for the CBN λ-calculus can be seen in Definition 2.1, and the syntax and semantics of the CBV λ-calculus can be seen in Definition 2.2 [16].

Definition 2.1 (Syntax and semantics of CBN)**.**

$$M, N ::= x \mid M\ N \mid \lambda x.M$$

$$\text{(Sub)}\ \frac{M \to M'}{M\ N \to M'\ N} \quad \text{(App)}\ \frac{}{(\lambda x.M)\ N \to M\{N/x\}}$$

Definition 2.2 (Syntax and semantics of CBV)**.**

$$M, N ::= V \mid M\ N$$
$$V ::= x \mid \lambda x.M$$

$$\text{(Sub)}\ \frac{M \to M'}{M\ N \to M'\ N} \quad \text{(Obj)}\ \frac{M\ \to M'}{V\ M \to V\ M'}$$

$$\text{(App)}\ \frac{}{(\lambda x.M)\ V \to M\{V/x\}}$$

The main difference between CBV and CBN appears in the case where the argument of an application is evaluated. While their expressive power is the same, their termination capabilities are not. In CBV the argument is evaluated before we perform the substitution, where as with CBN it uses a lazy evaluation, that only evaluates the expression when it is needed. This results in CBN and CBV having different termination for some expressions. An example of this is the λ-term $(\lambda x.y)((\lambda z.zz)(\lambda z.zz))$. Under the CBN semantics one can use application directly $(\lambda x.y)((\lambda z.zz)(\lambda z.zz)) \to y$. However, under the CBV semantics, one has to perform object reduction first.

$$\text{(Obj)}\ \frac{\text{(App)}\ \dfrac{}{((\lambda z.zz)(\lambda z.zz)) \to ((\lambda z.zz)(\lambda z.zz))}}{(\lambda x.y)((\lambda z.zz)(\lambda z.zz)) \to (\lambda x.y)((\lambda z.zz)(\lambda z.zz))}$$

One can rewrite the λ-term, such that it also terminates under the CBV semantics by writing arguments M as thunks $\lambda x.M$ where x does not occur free in M. This delays the computation until needed, at which point one supplies a bogus argement, forcing the computation inside the thunk. Rewriting in this way one can again use application directly $(\lambda x.y)(\lambda((\lambda z.zz)(\lambda z.zz))) \to y$. In this way one can essentially encode CBN semantics to CBV semantics. However, going the other way is a little more challenging, further using thunks will not work, instead one can use continuations. [16]

This motivates the call-by-push-value λ-calculus, which adds the necessary language constructs to represent both the CBN and CBV reduction strategies in an elegant way.

3 The call-by-push-value λ-calculus

Call-by-push-value λ-calculus (CBPV) was introduced in [12], and is a λ-calculus that encompasses both CBV and CBN. For the purposes of this paper, we use a reduced syntax of CBPV, that is still expressive enough to encode both the CBV, and the CBN λ-calculus elegantly. The syntax of this language, which we will refer to as CBPV, is shown in Definition 3.1. Note that unlike the original version of CBPV from [12], the version considered here is untyped.

Definition 3.1 (Syntax of CBPV)**.**

$$\begin{aligned} M, N &::= V \mid \lambda x.M \mid M\ V \mid \mathbf{force}\ V \mid \mathbf{return}\ V \mid M\ >\!\!>\!\!= \lambda x.N \\ V &::= x \mid \mathbf{thunk}\ M \end{aligned}$$

A key distinction is that between *expressions* $M, N \in \mathbf{Expr}$ and *values* $V \in \mathbf{Val}$. We let Var denote the countably infinite set of all variables, where $x, y, z, ... \in Var$. The binders of CBPV are $\lambda x.M$ that binds x in M and $M\ >\!\!>\!\!= \lambda x.N$ that binds x in N. We define substitution, free variables and bound variables in the usual way.

Our reduction semantics is shown in Definition 3.2. It is inspired by that of [17], though we omit the use of an unrolling function. This would unroll a term such as "$\mathbf{letrec}\ x = \lambda y.y\ \mathbf{in}\ N$". We do not need this, as we do not have mutual recursive aliasing, which would require the unroll operation.

Definition 3.2 (Reduction semantics of CBPV)**.**

$$\text{(Bind-1)}\ \frac{}{\mathbf{return}\ V >\!\!>\!\!= \lambda x.N \to N\{V/x\}} \qquad \text{(Bind-2)}\ \frac{M \to M'}{M >\!\!>\!\!= \lambda x.N \to M' >\!\!>\!\!= \lambda x.N}$$

$$\text{(App-1)}\ \frac{}{(\lambda x.M)\ V \to M\{V/x\}} \qquad \text{(App-2)}\ \frac{M \to M'}{M\ V \to M'\ V}$$

$$\text{(Force)}\ \frac{}{\mathbf{force}\ (\mathbf{thunk}\ M) \to M}$$

We now recall Levy's encodings of CBV and CBN in CBPV [13]. The two encodings use the same CBPV constructs, but in different ways. In the CBV encoding, computations are sequenced explicitly using **return** and $>\!\!>\!\!=$, so the argument of an application is evaluated before the function is applied. We use $\mathcal{V}[\![]\!]$ to denote the encoding of CBV to CBPV.

Definition 3.3 (Encoding CBV to CBPV)**.**

$$\begin{aligned} \mathcal{V}[\![x]\!] &= \mathbf{return}\ x \\ \mathcal{V}[\![\lambda x.M]\!] &= \mathbf{return}\ \mathbf{thunk}\ \lambda x.\mathcal{V}[\![M]\!] \\ \mathcal{V}[\![M\ N]\!] &= (\mathcal{V}[\![M]\!] >\!\!>\!\!= (\lambda f.\mathcal{V}[\![N]\!] >\!\!>\!\!= \lambda x.\mathbf{force}\ f\ x)) \end{aligned}$$

In the CBN encoding, evaluation is delayed instead. Variables are encoded as $\mathbf{force}\, x$, so the computation associated with a variable is forced only when the variable is used. In an application, the argument is wrapped in a thunk before it is passed to the function. We use $\mathcal{N}[\![]\!]$ to denote the encoding from CBN to CBPV.

Definition 3.4 (Encoding CBN to CBPV)**.**

$$\begin{aligned}\mathcal{N}[\![x]\!] &= \mathbf{force}\, x\\ \mathcal{N}[\![\lambda x.M]\!] &= \lambda x.\mathcal{N}[\![M]\!]\\ \mathcal{N}[\![M\ N]\!] &= \mathcal{N}[\![M]\!]\ \mathbf{thunk}\, \mathcal{N}[\![N]\!]\end{aligned}$$

Together, these two encodings make precise the informal observations from the previous section. For the term $(\lambda x.y)((\lambda z.zz)(\lambda z.zz))$, the encoded argument must still be evaluated in the CBV encoding before the body of the abstraction can be reached.

In the CBN encoding, the same argument is passed as a thunk. The application of $(\lambda x.y)$ therefore substitutes a delayed computation for x, and since x does not occur in y, the divergent argument is never forced. The difference between CBV and CBN is therefore represented in CBPV by the interaction between **thunk**, **force**, **return**, and $>\!\!>\!=$.

4 The Internal π-Calculus

In this section we will introduce the πI-calculus, [20] and discuss the motivations behind using to encode CBPV.

The π-calculus is a process calculus that can be seen as an extension to calculus of communicating systems (CCS) where names can be passed around, permitting the description of mobile systems. One can view πI-calculus as a variation of the π-calculus where free outputs have been removed and substitution replaced with alpha.conversion, or as a variation of CSS where alpha-conversion has been added. According to Sangiorgi the motivating factor in the creation of πI-calculus was to investigate calculi that lay somewhere in-between the π-calculus and CCS. Calculi that might be as expressive as the π-calculus, while still having the simplistic theory CCS. In what follows we will highlight how the πI-calculus simplifies much of the theory of the π-calculus, which secondarily should help emphasize the reasoning behind using the πI-calculus to encode CBPV. The expressive power of πI-calculus, should hopefully become self-evident throughout the paper.

Conceptually, processes in the πI-calculus may be understood as independent agents that interact by synchronising on names. For example, $a(x).P$ is the process that can receive a name on the channel a, and then continue as P, while the process $\overline{a}(b).Q$ is the process that can send a private name b on the channel a and then continue as Q. Consider the process

$$a(x).P \mid (\nu b)\overline{a}(b).Q$$

this represents two sub-processes that can communicate on a, after which the receiver may continue using the received name. The crucial restriction in the πI-calculus is that outputs are bound, meaning the name being sent is not a freely available external name, but one whose scope is controlled by the process. Replication gives rise to persistent behaviour. For instance, $!a(x).P$ is the process that can receive a name on the channel a arbitrarily many times, each time producing a copy of the continuation P with the received name bound to x. Links $x \to y$ provide a primitive form of forwarding, a process trying to interact on x can instead interact with a process on y, one might also think of it as a way to express renamings of names.

The choice of bisimulation in the π-calculus is to a large extent arbitrary. Many distinct definitions have been proposed including early-, late-, and open-bisimulation. Each reflecting

subtle differences in how input actions and substitutions are handled. Furthermore, most bisimulations for the π-calculus are not a congruence. However, in the case of the πI-calculus, this ambiguity disappears. Due to the duality of input and output actions, distinctions between early-, late-, and open-bisimulation collapse, and the definitions for them coincide. Even better, this definition of a bisimulation forms a congruence, making it a conceptually cleaner choice.

Another reasoning for the choice of using πI-calculus to encode CBPV in, was that we realised, while encoding CBPV to the π-calculus, that free outputs was not necessary. Since πI-calculus is a smaller calculus, since it has just removed free output, it is a strictly stronger result to encode it to πI-calculus than to the standard π-calculus.

The set of processes **Proc** denotes the set of processes built from the formation rules in Definition 4.1, where $a, b, c, x, y, z \cdots \in$ **Names**, a countable infinite set of names. The πI-calculus as it was originally presented by Sangiorgi in [20], varies slightly to what we present here, as we have opted to use replication and links rather than constant application. Had we chosen to use constant application, the constants we would need to encode CBPV would simply be replication and links. As such, for our purpose, one might simply embed these constants into the calculus itself. This seemed reasonable, especially since replication is a well established alternative to constant application in other variations of the π-calculus. We note that $(\vec{x})$ refers to a list of names, meaning $(\vec{x}) = (x_1, x_2, \ldots, x_n)$, and use $\vec{x}_1$ to denote the first element in the list, and $\vec{x}_n$ to be the last.

Definition 4.1 (Syntax of πI-calculus)**.**

$$P, Q \ ::= \ a(\vec{x}).P \mid \overline{a}(\vec{x}).P \mid (\nu x)P \mid \, !P \mid \vec{x} \to \vec{y} \mid P|Q \mid 0$$

By abuse of notation, we will usually remove any 0 suffix, which means that $\overline{a}(x)$ should be read as $\overline{a}(x).0$. In a similar manner we also omit parallel 0's, so $P \mid 0$ will be written as P. Lastly, we may also write $(\nu a, b)P$ to mean $(\nu a)(\nu b)P$. We define the notion of substitution ($\{x/y\}$), bound names (bn), free names (fn), subjects and objects in the usual way. We define names ($n(P)$) as the union of bound names and free names in P.

We present a labeled transition semantics, where transitions are of the form $P \xrightarrow{\mu} P'$. Labels $\mu \in$ **Action** are defined in Definition 4.2, and the set of valid transitions is given by the transition rules in Definition 4.3. For the sake of brevity, the symmetric versions of the (Com), (Par) and (Close) rules have been omitted. We denote the dual of an action μ by $\overline{\mu}$, $\overline{a(\vec{x})} = \overline{a}(\vec{x})$, $\overline{\overline{a}(\vec{x})} = a(\vec{x})$, and $\overline{\tau} = \tau$. The actions represent the observable behaviours of processes: $a(\vec{x})$ represents an input action where a process can receive a tuple of names on channel a, $\overline{a}(\vec{x})$ represents an output action where a process can send a tuple of names on channel a, and τ represents an internal, unobservable action resulting from communication between parallel processes.

Definition 4.2 (Actions in the πI-calculus)**.**

$$\mu ::= \overline{a}(\vec{x}) \mid a(\vec{x}) \mid \tau$$

Definition 4.3 (Labeled transition semantics for theπI-calculus)**.**

$$\text{(In)}\ \frac{}{a(\vec{x}).P \xrightarrow{a(\vec{x})} P} \qquad \text{(Out)}\ \frac{}{\overline{a}(\vec{x}).P \xrightarrow{\overline{a}(\vec{x})} P}$$

$$\text{(Par)}\ \frac{P \xrightarrow{\mu} P'}{P \mid Q \xrightarrow{\mu} P' \mid Q} bn(\mu) \cap fn(Q) = \emptyset$$

$$\text{(Com)}\ \frac{P \xrightarrow{\mu} P' \quad Q \xrightarrow{\overline{\mu}} Q'}{P \mid Q \xrightarrow{\tau} (\nu\vec{x})(P' \mid Q')} \mu \neq \tau, \vec{x} = bn(\mu)$$

$$\text{(Res)}\ \frac{P \xrightarrow{\mu} P'}{(\nu\vec{x})P \xrightarrow{\mu} (\nu\vec{x})P'} \vec{x} \cap n(\mu) = \emptyset$$

$$\text{(Link)}\ \frac{x \to y \mid x(\vec{a}).\overline{y}(\vec{b}).(\vec{b}_1 \to \vec{a}_1 \mid \ldots \mid \vec{b}_n \to \vec{a}_n) \xrightarrow{\mu} P'}{x \to y \xrightarrow{\mu} P'}$$

$$\text{(Rep)}\ \frac{P \mid !P \xrightarrow{\mu} P'}{!P \xrightarrow{\mu} P'} \qquad \text{(Alpha)}\ \frac{P \equiv_\alpha Q \quad Q \xrightarrow{\mu} P'}{P \xrightarrow{\mu} P'}$$

Compared to the π-calculus with which some readers may be more familiar, an interesting aspect about the πI-calculus is that communication is now an agreement on the choice of name. By this we mean an output action cannot simply send a name to be used in a substitution in another process, in fact as some might have noticed, there is no substitution. Instead, as seen in the (Alpha) rule, alpha conversion can be used to make processes agree on a name. The processes $a(x).x(u)$ and $\overline{a}(y).\overline{y}(v)$ can communicate, by using the (Alpha) and (Com) rules to agree on a name. By alpha converting both x and y to the name z, one gets the process $(\nu z)(z(u) \mid \overline{z}(v))$.

Another rule that stands out is the (Link) rule. The idea behind a link is that when you send a name to a link, a pointer to the received name is sent.

We will write $\Rightarrow$ as the transitive and reflexive closure of $\xrightarrow{\tau}$, and write $P \stackrel{\mu}{\Rightarrow} P'$ to mean $P \Rightarrow Q \xrightarrow{\mu} Q' \Rightarrow P'$.

A useful definition is that of barbs. Processes having no outgoing barbs is a notion that will come up once we present Theorem 1.

Definition 4.4 (Barbs)**.** Let P be a process. We say that P has an outgoing barb on a, written $P \downarrow_{\overline{a}}$, if P can immediately perform an output action on the channel a. That is, there exist names $\vec{x}$ and a process P' such that $P \xrightarrow{\overline{a}(\vec{x})} P'$.

Similarly, we say that P has an incoming barb on a, written $P \downarrow_a$, if P can immediately perform an input action on the channel a. That is, there exist names $\vec{x}$ and a process P' such that $P \xrightarrow{a(\vec{x})} P'$.

We say that P has no outgoing (incoming) barbs if $\forall a.\neg P \downarrow_{\overline{a}}$ ($\forall a.\neg P \downarrow_a$).

In the πI-calculus, early, late, and open bisimilarity coincide, and bisimilarity is a congruence. This removes one of the usual choices involved in selecting a behavioural equivalence for the target calculus. [21] We opt to use weak bisimulation as it abstracts from internal computation steps. Since one CBPV reduction may be simulated by several internal τ-transitions in the

πI-calculus, we compare processes by allowing such internal steps before and after observable actions. The observable actions are the process barbs, namely its externally visible communication capabilities.

Definition 4.5 (Weak bisimulation)**.** A symmetric relation $\mathcal{R}$ on πI-calculus processes is a weak bisimulation if $P\mathcal{R}Q$ implies:

- whenever $P \Rightarrow P'$, there exists a Q' s.t. $Q \Rightarrow Q'$ and $P'\mathcal{R}Q'$.
- whenever $P \Rightarrow^{\mu} P'$, where $\mu \neq \tau$ and $bn(\mu) \cap fn(Q) = \emptyset$, there exists a Q' s.t. $Q \Rightarrow^{\mu} Q'$ and $P'\mathcal{R}Q'$.

We say that two processes P and Q are weakly bisimilar, written $P \approx Q$, if $P\mathcal{R}Q$, for some weak bisimulation $\mathcal{R}$.

We will use some bisimilarity laws to rewrite processes while preserving bisimilarity. The rules relevant to this paper are presented in Lemma 4.1; they serve both to highlight the interesting aspects of processes and to justify the use of appropriate lemmas.

Lemma 4.1 (Bisimilarity laws)**.**

$$(\nu x)(P \mid Q) \approx P \mid (\nu x)Q \quad \text{if } x \notin \mathrm{fn}(P)$$

$$(\nu x)0 \approx 0$$

$$(\nu y)(!y(x).P) \approx 0$$

$$(\nu y)(\overline{y}x.P) \approx 0$$

$$(\nu y)(\nu x)P \approx (\nu x)(\nu y)P$$

$$(\nu x)(P \mid (x \to y)) \approx P\{y/x\} \quad \text{if } y \notin \mathrm{fn}(P)$$

We have omitted the proofs of the bisimilarity laws and instead refer to [27], as they are straightforward to prove.

Another calculus in which the bisimulation choice becomes clear, is with the asynchronous π-calculus (Aπ), where ground, late and open bisimulation coincide. [27] In fact for many of the arguments made for πI, analogous arguments can be made for Aπ. However, we argue that πI is better suited to work with de Bruijn indices [7], and formalizations where they are used. de Bruijn indices is a well known way to formalize binders in proof assists such a Rocq. They work by replacing bound occurances of names with indices pointing to the binder that they are referencing. To handle free occurences various alterations exists, in the locally nameless approach free occurrences are still represented with names. As such the process $a(x).x(y).x(z)|\overline{a}(x).\overline{x}(y).\overline{x}(z)$, can be represented in a locally nameless manner as $a().0().1()|\overline{a}().\overline{0}().\overline{1}()$. An advantage of the de Bruijn indices is that alpha equivalence comes for free, in the sense that two alpha equivalent terms must also be syntactically equivalent. This plays very nicely with πI as communication is based on being able to find an agreement using the (Alpha) rule, and by extension alpha conversion. It turns out that when using de Bruijn indices the (Alpha) rule becomes redundant and communication between two processes only occurs if they agree on a syntactic level. This may sound like one losses some expresive power, as one can not simply use alpha conversion to

make processes agree on a name. However, when using de Bruijn indices one has to keep track of indices and correct them when certain transitions occur, by doing this alpha equivalent terms will always agree. Another benefit of πI in this regard is that it does not use substitution, as this further complicates this tracking of indices. To summarize πI is well suited for the use of de Bruijn indices exactly because it relies on alpha conversion and not substitution, two things that cannot be said about Aπ.

5 Encoding CBPV in the πI-calculus

With the πI-calculus introduced, we proceed by encoding CBPV in the πI-calculus, as seen in Definition 5.1. Since the polyadic πI-calculus can easily be encoded in the monadic πI-calculus without loss of generality [20], we gain an encoding in the monadic πI-calculus for free. One should be aware that in the polyadic πI-calculus, links can communicate with outputs with varying arity.

Definition 5.1 (Encoding of CBPV in πI-calculus)**.** For two distinct names u, r and a CBPV term M, where $u, r \notin n(M)$ the encoding of M in the πI-calculus is defined as:

$$
\begin{aligned}
[\![\lambda x.M]\!]^r_u &= \overline{u}(e).e(u,r,x).[\![M]\!]^r_u && e \notin fn(M)\\
[\![M\ V]\!]^r_u &= (\nu a,b,c,d)(\\
&\qquad a(e).c(x).\overline{e}(u',r',x') \\
&\qquad\quad .(u' \to u \mid r' \to r \mid x' \to x) \\
&\qquad \mid [\![M]\!]^b_a \\
&\qquad \mid [\![V]\!]^d_c \\
&\quad) && a,b,c,d \notin fn(M) \cup fn(V)\\
[\![\mathbf{force}\ V]\!]^r_u &= (\nu a,b)([\![V]\!]^b_a \mid a(y).\overline{y}(u',r').(u' \to u | r' \to r)) && a,b \notin fn(V)\\
[\![\mathbf{return}\ V]\!]^r_u &= [\![V]\!]^u_r \\
[\![M >\!\!>\!\!= \lambda x.N]\!]^r_u &= (\nu a,b)([\![M]\!]^b_a \mid b(x).[\![N]\!]^r_u) && a,b \notin fn(M) \cup fn(N)\\
[\![V]\!]^r_u &= (\overline{u}(y).[\![y := V]\!]) && y \notin fn(V)\\
[\![y := x]\!] &= !y(u,r).\overline{x}(u',r').(u' \to u | r' \to r) \\
[\![y := \mathbf{thunk}\ M]\!] &= !y(u,r).[\![M]\!]^r_u
\end{aligned}
$$

The encoding works by simulating each reduction in CBPV, with multiple communications in the πI-calculus. The main difference between the two calculi is that in CBPV you can substitute whole values, while in πI-calculus, you can only agree on names. Therefore, to simulate the substitution of a value, we use "pointers", which can be seen with the notation $[\![y := V]\!]$. One way to understand this is, that the name y becomes a name that give access to a process that can simulate V. We utilize two handles, as there are two different constructs in CBPV, that can substitute, namely application and binding, and in the πI-calculus, we need to be able to distinguish between the two. Therefore, u is used for applications, and r is the return handle, used for binding. Notably, $[\![y := x]\!]$ is encoded as a way to send a force to the name of the variable, where a $[\![y := \mathbf{thunk}\ M]\!]$ is encoded as a replication of the encoding of M. The intuition is, that a force will unlock one copy of the encoding of M.

A small example to help the reader gain intuition behind the encoding, we look at the encoding of **force thunk** M. If we encode this, we gain the following process:

$$\begin{aligned}
[\![\textbf{force thunk}\ M]\!]^r_u = \ & (\nu a,b)(\overline{a}(y).!y(u,r).[\![M]\!]^r_u \mid a(y).\overline{y}(u',r').(u' \to u | r' \to r)) \\
\to^\tau \ & (\nu a,b)(\nu y)(!y(u,r).[\![M]\!]^r_u \mid \overline{y}(u',r').(u' \to u | r' \to r)) \\
\to^\tau \ & (\nu a,b)(\nu y)(!y(u,r).[\![M]\!]^r_u | [\![M]\!]^{r'}_{u'} \mid (u' \to u | r' \to r)) \\
\approx \ & (\nu u',r')([\![M]\!]^{r'}_{u'} \mid (u' \to u | r' \to r))
\end{aligned}$$

The difference from the encoding of M, and what we reduce down to, is essentially that the handles u and r in M have been replaced with u' and r', but through the links, these can be used to communicate on u and r. In the πI-calculus, multiple reduction paths may exist, and different reductions can be applied at different times, leading to variation in the sequence of computational steps. Despite this non-determinism in reduction, our encoding satisfies a confluence property: all reduction paths ultimately lead to the same result (up to multiple unfoldings of replications and links). In fact, at any given moment only a single reduction path is possible in our encoding, just like in the CBPV semantics (again, up to multiple unfoldings of replication and links).

Our encoding has an operational correspondence with the original terms in CBPV.

Theorem 1 (Operational Correspondence)**.** *Let M, N be terms in CBPV, P, P' be processes in the πI-calculus, and u, r be distinct names s.t. $u, r \notin FV(M) \cup FV(N)$. Then*:

1. (*Soundness*) *if $M \to N$ then $\exists P.[\![M]\!]^r_u \Rightarrow P$ and $P \approx [\![N]\!]^r_u$.*
2. (*Completeness*) *$\forall P$. If $[\![M]\!]^r_u \Rightarrow P$, then $\exists N, P'.P \Rightarrow P'$ and $P' \approx [\![N]\!]^r_u$, and $M \leftrightarrows N$. Furthermore, if $P \neq P'$ then $\forall a.\neg P \downarrow_{\overline{a}} \wedge \neg P \downarrow_a$.*

Theorem 1 expresses that the encoding must be both sound (1) and complete (2). The keen reader may notice that our notions of soundness and completeness are swapped compared some parts of the litterature. We use the term soundness to denote that every reduction of a CBPV term is simulated by reductions of its encoding. Conversely, completeness to denote that every reduction sequence of an encoded process leads to a process, which matches a reduction between CBPV terms. Further, we also require from our notion of completeness, that any intermediate process has no barbs, but can only perform internal reduction steps until it reaches a process that is bisimilar with an encoding of a term.

5.1 Proving soundness and completeness

In this section we sketch the proof of Theorem 1. The proof of soundness is done by induction the reduction $M \to N$, where we will highlight the case of (App-1) and outline the rest. The proof of completeness is done by induction on the struture of an encoded term M, similarly we alsa highlight a case here namely application and outline the rest. For a full proof we refer to [3].

Essential to the proof is a substitution lemma that explains how substitution of values in CBPV is mirrored as name substitution in the πI-calculus.

Lemma 5.1 (Substitution)**.** Let M be a CBPV term, V a value, x a variable, and u, r distinct names. Then:

$$[\![M\{V/x\}]\!]^r_u \approx (\nu y)([\![M]\!]^r_u\{y/x\} | [\![y := V]\!])$$

Proof of Lemma 5.1. using induction in the structure of M and makes use of the "up-to-bisimilarity" technique, using the bisimilarity laws presented in 4.1, and in particular that one can move the restriction (νy) and $[\![y := V]\!]$ further inside M while still maintaining bisimilarity with the encoding of $M\{y/x\}$. The proof then comes down to the base case where $M = x$, which can be proven as a simple bisimilarity. □

Proof of soundness (1). The proof of soundness, (1), uses induction on the reduction $M \to N$. This means that for each rule with which $M \to N$ can be concluded, we need to show that (1) holds. We look at the case for the (App-1)-rule, and provide sketches for the remaining cases.

For the (App-1)-rule, we have:

$$\text{(App-1)}\ \frac{}{(\lambda x.M)\ V \to M\{V/x\}}$$

For (1) to hold, it must be the case that $[\![(\lambda x.M)\ V]\!]^r_u$ can reduce to a process that is bisimilar to $[\![M\{V/x\}]\!]^r_u$.

We begin with unfolding the encoding:

$$\begin{aligned}[\![(\lambda x.M)\ V]\!]^r_u =& (\nu a,b,c,d)(a(e).c(x).\overline{e}(u',r',x').(u' \to u | r' \to r | x' \to x) \\ & | \ \overline{a}(e).e(u',r',x).[\![M]\!]^{r'}_{u'} \mid [\![V]\!]^d_c)\end{aligned}$$

From this, we get the following reduction:

$$\begin{aligned}& (\nu a,b,c,d)(a(e).c(x).\overline{e}(u',r',x').(u' \to u | r' \to r | x' \to x) \\ & | \ \overline{a}(e).e(u,r,x).[\![M]\!]^r_u \mid [\![V]\!]^d_c) \\ \to^\tau & (\nu e)(\nu a,b,c,d)(c(x).\overline{e}(u',r',x').(u' \to u | r' \to r | x' \to x) \\ & | \ \ e(u,r,x).[\![M]\!]^r_u \mid [\![V]\!]^d_c) \\ \to^\tau & (\nu e)(\nu y)(\nu a,b,c,d)(\overline{e}(u',r',x').(u' \to u | r' \to r | x' \to y) \\ & | \ \ e(u,r,x).[\![M]\!]^r_u \mid [\![y := V]\!]) \\ \to^\tau & (\nu e)(\nu y)(\nu u',r',x')(\nu a,b,c,d)((u' \to u | r' \to r | x' \to y) \\ & | \ \ [\![M]\!]^r_u\{u'/u,r'/r,x'/x\} \mid [\![y := V]\!])\end{aligned}$$

We now prove that

$$\begin{aligned}& (\nu e)(\nu y)(\nu u',r',x')(\nu a,b,c,d) \\ & \quad ((u' \to u | r' \to r | x' \to y) \mid [\![M]\!]^r_u\{u'/u,r'/r,x'/x\} \mid [\![y := V]\!]) \\ & \approx [\![M\{V/x\}]\!]^r_u\end{aligned}$$

We prove this bisimilarity using up-to-bisimilarity. First, we can remove the restriction $(\nu a,b,c,d)$ and (νe) using the bisimilarity laws, as there are no longer any occurances of a, b, c, d or e.

$$\begin{aligned}& (\nu e)(\nu y)(\nu u',r',x')(\nu a,b,c,d)((u' \to u | r' \to r | x' \to y) \mid [\![M]\!]^r_u\{u'/u,r'/r,x'/x\} \mid [\![y := V]\!]) \\ & \approx (\nu y)(\nu u',r',x')((u' \to u | r' \to r | x' \to y) \mid [\![M]\!]^r_u\{u'/u,r'/r,x'/x\} \mid [\![y := V]\!])\end{aligned}$$

We now use the bisimilarity laws, to replace the links with substitutions:

$$\begin{aligned}(\nu y)(\nu u', r', x')((u' \to u | r' \to r | x' \to y) \mid [\![M]\!]^r_u\{u'/u, r'/r, x'/x\} \mid [\![y := V]\!]) \\ \approx (\nu y)([\![M]\!]^r_u\{y/x\} \mid [\![y := V]\!])\end{aligned}$$

We use Lemma 5.1 to conclude the following:

$$(\nu y)([\![M]\!]^r_u\{y/x\} \mid [\![y := V]\!]) \approx [\![M\{V/x\}]\!]^r_u)$$

With this, we have shown that we can match the reduction in CBPV with 4 reductions in the πI-calculus, and therefore proved (1), when the reduction is concluded using the (App-1)-rule.

For the (Bind-1)-rule, where $\mathbf{return}\, V \gg\!= \lambda x.N \to N\{V/x\}$, we unfold the encoding of the bind and return constructs. The encoding of $\mathbf{return}\, V \gg\!= \lambda x.N$ is:

$$\begin{aligned}[\![\mathbf{return}\, V \gg\!= \lambda x.N]\!]^r_u =& (\nu a, b)([\![\mathbf{return}\, V]\!]^b_a \mid b(x).[\![N]\!]^r_u) \\ =& (\nu a, b)([\![V]\!]^a_b \mid b(x).[\![N]\!]^r_u)\end{aligned}$$

This reduces via communication on channel b to a process where V is substituted for x in N. We then apply Lemma 5.1 to show that the resulting process is bisimilar to $[\![N\{V/x\}]\!]^r_u$. The key steps involve unfolding the encoding of V, performing the substitution via links, and using the bisimilarity laws to clean up the resulting process.

For the (Force)-rule, where $\mathbf{force}\,(\mathbf{thunk}\, M) \to M$, we unfold the encoding of force and thunk. The encoding of $\mathbf{force}\,(\mathbf{thunk}\, M)$ is:

$$\begin{aligned}[\![\mathbf{force}\,(\mathbf{thunk}\, M)]\!]^r_u =& (\nu a, b)([\![\mathbf{thunk}\, M]\!]^b_a \mid a(y).\overline{y}(u', r').(u' \to u | r' \to r)) \\ =& (\nu a, b)((\nu y)\overline{a}y.!y(u, r).[\![M]\!]^r_u \mid a(y).\overline{y}(u', r').(u' \to u | r' \to r))\end{aligned}$$

This reduces via communication on channel a to a process where the thunk is unlocked, and the resulting process is bisimilar to $[\![M]\!]^r_u$. The reduction sequence involves unfolding the replication and links, and then using the bisimilarity laws to show that the resulting process matches the encoding of M.

For the evolutionary rules (App-2) and (Bind-2), where $M \to M'$, the proof follows by induction. We use the inductive hypothesis to show that $[\![M]\!]^r_u$ reduces to a process bisimilar to $[\![M']\!]^r_u$. For (App-2), we unfold the encoding of the application and use the inductive hypothesis to show that the reduction of M can be extended to the full application. For (Bind-2), we similarly use the inductive hypothesis to show that the reduction of M in the bind construct results in a process bisimilar to the encoding of $M' \gg\!= \lambda x.N$. □

Proof of completeness (2). The proof of completeness proceeds by structural induction on the encoded term M. Here we focus on the case where M is an application, i.e., $M = M'\ V'$, and sketch the proofs for the remaining cases.

We begin with the encoding of $M'\ V'$:

$$\begin{aligned}[\![M'\ V']\!]^r_u =& (\nu a, b, c, d)(a(e).c(x).\overline{e}(u', r', x').(u' \to u | r' \to r | x' \to x) \\ & \mid [\![M']\!]^b_a \mid [\![V]\!]^d_c)\end{aligned}$$

For this process to reduce, there are two possibilities:

1. **Internal reduction in $[\![M']\!]^b_a$:** If $[\![M']\!]^b_a$ performs an internal τ-transition, by the inductive hypothesis, it will eventually reach a process $[\![N]\!]^b_a$ that is weakly bisimilar to the encoding of some CBPV term N, without having any barbs. Thus, the entire process $[\![M'\ V']\!]^r_u$ will similarly reduce to a state bisimilar to $[\![N\ V']\!]^r_u$ without external communication.
2. **Communication with $a(e)$:** If $[\![M']\!]^b_a$ can output on channel a, then by inspection of the encoding rules, M' must be an abstraction, i.e., $M' = \lambda x.M''$. In this case, the reduction proceeds as follows:

   The input $a(e)$ communicates with $[\![\lambda x.M'']\!]^b_a = a(e).e(u).e(r).e(x).[\![M'']\!]^r_u$, resulting in:

   $$\begin{aligned}(\nu a,b,c,d)(&e(u).e(r).e(x).[\![M'']\!]^r_u \\ &\mid \overline{e}(u',r',x').(u' \to u | r' \to r | x' \to x) \\ &\mid [\![V]\!]^d_c)\end{aligned}$$

   This further reduces through a series of τ-transitions to:

   $$(\nu y)([\![M'']\!]^r_u\{y/x\} | [\![y := V]\!])$$

   By Lemma 5.1, this process is weakly bisimilar to $[\![M''\{V/x\}]\!]^r_u$, which corresponds to the reduction $(\lambda x.M'')\ V' \to M''\{V'/x\}$ in CBPV.

   Crucially, in all intermediate steps, the channel e remains restricted, so the process cannot communicate with its surroundings until it reaches a state bisimilar to an encoding of a CBPV term.

In both cases, we have shown that $[\![M'\ V']\!]^r_u$ reduces to a process bisimilar to the encoding of a CBPV term, without external communication, thus proving completeness for applications.

For the remaining cases:

- **Force:** If $M = \mathbf{force}\, V'$, the encoding sets up a communication that either unlocks a thunk (if $V' = \mathbf{thunk}\, M''$) or forwards a variable (if $V' = x$). In both cases, the reduction sequence eventually reaches a process bisimilar to $[\![M'']\!]^r_u$ or the encoding of the value that x refers to, respectively.
- **Return:** If $M = \mathbf{return}\, V'$, the encoding simply swaps the handles, and the process reduces directly to a state bisimilar to $[\![V']\!]^u_r$, which corresponds to the terminal state of $\mathbf{return}\, V'$.
- **Bind:** If $M = M' \gg\!= \lambda x.N'$, the encoding sets up a communication channel between $[\![M']\!]^b_a$ and the continuation $b(x).[\![N]\!]^r_u$. By the inductive hypothesis, $[\![M']\!]^b_a$ reduces to a process bisimilar to $[\![\mathbf{return}\, V]\!]^b_a$ for some value V. This then communicates with the continuation to produce a process bisimilar to $[\![N\{V/x\}]\!]^r_u$, matching the CBPV reduction $M' \gg\!= \lambda x.N' \to N\{V/x\}$.

In all cases, the process cannot communicate with its surroundings until it reaches a state bisimilar to an encoding of a CBPV term. □

6 Gorla's properties of encodings

In [9], Daniele Gorla presents 5 properties to evaluate the quality of encodings between process calculi. These properties are also relevant for our encoding, even if CBPV is not a process calculus of this kind, and we here formulate a version of Gorla's properties in Definition 6.1.

We let $M \mapsto^{\omega}$ denote divergence, i.e. that M has an infinite reduction sequence and let $P \to^{\omega}$ denote the same property for processes.

For us to be able to define all of the properties, we also need to define what it means for a term to be successful. For a CBPV term, we define this to be a value. In the πI-calculus, a successful term is one that has an outgoing barb on the handle u to a pointer x, and having a subprocess $[\![x := V]\!]$, that is not restricted on x.

Definition 6.1 (Properties of encodings)**.** Let u, r be distinct and free in the corresponding term, then an encoding $[\![\cdot]\!]^r_u$ of CBPV in the πI-calculus, should satisfy the following:

1. (Compositionality) An encoding $[\![\cdot]\!]^r_u$ is compositional if, for every term constructor o of terms in CBPV, there exists a context C and a set of names $N = \{n_1, \dots n_m\}$, such that $[\![o(M_1, \dots, M_k]\!]^r_u = C[[\![M_1]\!]^{n_2}_{n_1}; \dots; [\![M_k]\!]^{n_m}_{n_{m-1}}]$.
2. (Name invariance) An encoding $[\![\cdot]\!]^r_u$ is name invariant, if, for every term M, and every substitution of variables σ in CBPV, we have a corresponding substitution σ' in the πI-calculus, such that $[\![\sigma(M)]\!] \approx \sigma'([\![M]\!])$.
3. Operational correspondence) An encoding $[\![\cdot]\!]^r_u$ is operational correspondent if it is:
   - (Sound): For all $M \Rightarrow N$, $[\![M]\!]^r_u \Rightarrow P$ and $P \approx [\![N]\!]^r_u$.
   - (Complete): For all P where $[\![M]\!]^r_u \Rightarrow P$, there exists an N so that $M \Mapsto N'$ and $P \Rightarrow P'$ and $P' \approx [\![N]\!]^r_u$
4. (Divergence reflection) An encoding $[\![\cdot]\!]^r_u$ reflects divergence, if for every M where $[\![M]\!]^r_u \to^{\omega}$, it holds that $M \mapsto^{\omega}$.
5. (Success sensitiveness) An encoding $[\![\cdot]\!]^r_u$ is success sensitive if, for every M, it holds that $M \Mapsto V$ if and only if $\exists y.[\![M]\!]^r_u \Rightarrow P$ and $P \xrightarrow{\overline{y}v} P'$.

We now proceed with a proof sketch for each criteria, proving that our encoding sitsifies Gorla's criteria.

Compositionality Our encoding is compositional. For every term constructor o of terms in CBPV, there exists a context C and a set of names $N = \{n_1, \dots n_m\}$, such that $[\![o(M_1, \dots, M_k)]\!]^r_u = C[[\![M_1]\!]^{n_2}_{n_1}; \dots; [\![M_k]\!]^{n_m}_{n_{m-1}}]$. This follows directly from the definition of our encoding (Definition 5.1), where each constructor is encoded as a process context surrounding the encodings of its subterms. For example, the encoding of an application $[\![M\ V]\!]^r_u$ is defined as a parallel composition of the encodings of M and V within a restricted context.

Name invariance Our encoding is name-invariant. For any CBPV term M and substitution σ of variables in CBPV, there exists a corresponding substitution σ' in the πI-calculus such that $[\![\sigma(M)]\!]^r_u \approx \sigma'([\![M]\!]^r_u)$. This holds because our encoding uses fresh names for internal communication (e.g., a, b, c, d in the encoding of applications) and preserves the structure of free variables. The handles u and r are treated uniformly, and substitutions in CBPV translate to name substitutions in the πI-calculus without altering the behavioural equivalence of the encoded process.

Operational correspondence Our encoding satisfies operational correspondence, which consists of two sub-properties: soundness and completeness, as proven in Theorem 1.

Divergence reflection To prove that our encoding satisfies divergence reflection, we prove the contrapositive: if $M \not\Uparrow^{\omega}$, then $[\![M]\!]^r_u \not\Uparrow^{\omega}$. If M does not diverge, it reduces to a terminal value in n steps. We show that $[\![M]\!]^r_u$ reduces to a final process within a bounded number of steps. From the completeness proof, we know that no infinite reductions are introduced; each process terminates in finitely many steps to a process weakly bisimilar to an encoding of a term. The soundness proof shows that each reduction in CBPV can be simulated by a finite number of steps in the πI-calculus. While each use of Lemma 5.1 may hide an extra reduction (due to introduced links), the outermost level requires at most 4 reductions to match a CBPV reduction, with future reductions potentially requiring up to 1 additional step. For terms like $\mathbf{force}\, x$, there are 2 reductions in the πI-calculus even though $n = 0$. Accounting for these, we bound the number of πI-calculus reductions to reach a final process by $(4+n)\cdot n+(n+2)$, where n is the number of CBPV reductions. This establishes that $[\![M]\!]^r_u \to^{\omega}$ implies $M \mapsto^{\omega}$.

Success sensitiveness Our encoding is success-sensitive. A CBPV term M reduces to a value V if and only if $[\![M]\!]^r_u$ reduces to a process that can output a pointer on the return handle u. Specifically, $M \Mapsto \mathbf{return}\, V$ if and only if $\exists y.[\![M]\!]^r_u \Rightarrow P$ and $P \xrightarrow{\overline{u}y} P'$. By inspection of the encoding, this is indeed the case: in all other scenarios, either restrictions prevent subprocesses from outputting on the u handle, or such an output is behind a prefix. Furthermore, the soundness and completeness proofs demonstrate that an encoding only reduces to a process weakly bisimilar with the encoding of a value if the original term reduces to a value.

7 Correspondence with the original encodings by Milner

In [14], Milner encodes both CBV and CBN in the π-calculus. We do not claim that our encoding reproduces those processes syntactically, but there is a close behavioural correspondence once Levy's encodings into CBPV are composed with our encoding into the π-calculus. In particular, the variable and abstraction clauses line up closely in both the CBN and CBV cases, while the extra handle structure of our πI-calculus translation explains the remaining mismatch.

For reference, Milner's direct encodings are as follows.

Definition 7.1 (Milner's encoding of the CBN λ-calculus)**.**

$$\begin{aligned}\mathcal{L}[\![x]\!]_u &= \overline{x}u\\ \mathcal{L}[\![\lambda x.M]\!]_u &= u(x).u(q).\mathcal{L}[\![M]\!]_q\\ \mathcal{L}[\![M\ N]\!]_u &= (\nu q)(\mathcal{L}[\![M]\!]_q \mid q(x,r).(!x(s).\mathcal{L}[\![N]\!]_s \mid \overline{r}u))\end{aligned}$$

Definition 7.2 (Milner's encoding of the CBV λ-calculus)**.**

$$\begin{aligned}\mathcal{E}[\![x]\!]_p &= \overline{p}x\\ \mathcal{E}[\![\lambda x.M]\!]_p &= (\nu y)(\overline{p}y\ !y(x,q).\mathcal{E}[\![M]\!]_q)\\ \mathcal{E}[\![M\ N]\!]_p &= (\nu q)(\mathcal{E}[\![M]\!]_q \mid q(f).(\nu r)(\mathcal{E}[\![N]\!]_r \mid r(v).\overline{f}v,p))\end{aligned}$$

The similarities can be seen most clearly in the variable cases. For CBN, Milner's encoding forwards the variable directly on the result channel, whereas our double encoding first forces the variable through a CBPV thunk and then exposes it on the two handles used by our πI-calculus encoding. After the intermediate reductions, the resulting process has the same shape as Milner's direct encoding, up to the extra handle.

For instance, the CBN variable case gives

$$[\![\ \mathcal{N}[\![x]\!]\ ]\!]_u^r = [\![\mathbf{force}\ x]\!]_u^r \approx (\nu s)(\overline{x}s \mid \overline{s}u.\overline{s}r),$$

which is the same direct forwarding pattern as Milner's encoding, but with the additional handle used by our πI-calculus translation.

The same pattern appears for CBV. Variables are still passed through with minimal change, but abstractions acquire the additional thunk and return structure introduced by Levy's encoding of CBV into CBPV. The resulting πI-calculus process still follows Milner's pointer-passing shape, but with the extra structure required by the intermediate CBPV layer.

For CBV, the variable case is even closer:

$$[\![\ \mathcal{V}[\![x]\!]\ ]\!]_u^r = [\![\mathbf{return}\ x]\!]_u^r = [\![x]\!]_r^r,$$

so the only difference from the direct Milner-style variable encoding is that the intermediate translation uses the second handle. By contrast, abstractions are necessarily larger because

$$[\![\lambda x.M]\!]^v = \mathbf{return\ thunk}\ \lambda x.[\![M]\!]^v,$$

so the resulting πI-calculus term reflects the additional thunking and return steps introduced by the CBPV layer.

Thus the composition of Levy's encodings with ours, does not lead to literal process equality with Milner's encodings. Rather, it is an explicit correspondence at the level of behaviour. The direct encodings and the CBPV route lead to processes that behave in closely related ways, and the remaining differences are explained by the intermediate translation.

8 Conclusion

In this chapter, we summarize the results of our work and discuss future work, that could follow the results presented in this paper.

8.1 Results

We have presented an encoding of a reduced version of CBPV in the πI-calculus, and proven that this encoding is both sound and complete. Our encoding demonstrates that the πI-calculus is expressive enough to capture CBPV, unifying call-by-value and call-by-name in a single process calculus framework. Further, it simplifies formal reasoning due to πI-calculus's clean bisimulation theory.

We adapted Gorla's properties [9] for a good encoding to fit within the context of CBPV, and proved that our encoding satisfies these properties. This means that our encoding closely simulates the behaviour of CBPV, with both divergence and termination.

Lastly, when composing with Levy's encoding [12] of CBN and CBV in CBPV, with our encoding of CBPV in πI-calculus, there seems to be a behavioural correspondence the encodings introduced by Milner in [14]. Meaning, if we instead encode CBV or CBN to CBPV, and then use our encoding to encode in the πI-calculus, we would reach a process that behaves similarly to the process obtained by directly encoding CBV or CBN in the π-calculus.

8.2 Future work

Part of the reason for using πI-calculus was it relationship with formalizations, especially when using De Bruijn indices. Future work would include a formalization of the current results. In the extended work we have already begun to formalize the results in Rocq. The formalization in still in the early stages, much of the skeletons of the proofs have been been made an proven, though with a dependency on a lot of lemmas that still remain to be proven.

Another approach to extend the work in this paper could be to extend our encoding, to also include more of the original constructs in Levy's CBPV. These constructs allow more advanced CBN and CBV to be encoded in the CBPV, and then with an extended encoding of the CBPV, a unified encoding of these extended paradigms could then be made in the πI-calculus.

One could also look into typing our encoding. We suspect that there is a possible correspondence between the type system of CBPV, and a receptiveness type system for the π-calculus, such as the one presented in [24], as well as a polyadic type system. We believe this, since the places where there would multiple outputs on the same channel, is in cases such as $[\![V\ V']\!]^r_u$, which would also be ill-typed in CBPV.

Lastly, a nice property of an encoding is also that it preserves equivalence. In [18], they present a formal equational theory for CBPV. We conjecture that if two CBPV term M, N are bisimilar, then $[\![M]\!]^r_u \approx [\![N]\!]^r_u$. However, we leave it as future work to actually prove that this holds.

References

[1] Beniamino Accattoli, Horace Blanc & Claudio Sacerdoti Coen (2023): *Formalizing Functions as Processes*. In Adam Naumowicz & René Thiemann, editors: *14th International Conference on Interactive Theorem Proving (ITP 2023)*, *Leibniz International Proceedings in Informatics (LIPIcs)* 268, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, pp. 5:1–5:21, doi:10.4230/LIPIcs.ITP.2023.5. Available at `https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2023.5`.

[2] J. W. Backus, F. L. Bauer, J. Green, C. Katz, J. McCarthy, A. J. Perlis, H. Rutishauser, K. Samelson, B. Vauquois, J. H. Wegstein, A. van Wijngaarden, M. Woodger & Peter Naur (1960): *Report on the algorithmic language ALGOL 60*. *Commun. ACM* 3(5), p. 299–311, doi:10.1145/367236.367262. Available at `https://doi.org/10.1145/367236.367262`.

[3] Benjamin Bennetzen, Nikolaj Rossander Kristensen & Peter Buus Steffensen (2025): *Encoding call-by-push-value in the pi-calculus*. arXiv:2506.10584.

[4] Benjamin Bennetzen, Nikolaj Rossander Kristensen & Peter Buus Steffensen (2025): *Formalisation of Pi-I-calculus, CPBV and our encoding*. Available at `https://github.com/BenjaminCB/cbpv-to-pi-coq`.

[5] Michele Boreale & Davide Sangiorgi (1995): *A fully abstract semantics for causality in the π-calculus*. In: *Annual Symposium on Theoretical Aspects of Computer Science*, pp. 243–254.

[6] Arthur Charguéraud (2012): *The Locally Nameless Representation*. *Journal of Automated Reasoning* 49(3), pp. 363–408. Available at `https://www.proquest.com/scholarly-journals/locally-nameless-representation/docview/1366364526/se-2`. Copyright - Springer Science+Business Media B.V. 2012; Last updated - 2023-11-27.

[7] N.G de Bruijn (1972): *Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem*. Indagationes Mathematicae

(Proceedings) 75(5), pp. 381–392, doi:10.1016/1385-7258(72)90034-0. Available at https://www.sciencedirect.com/science/article/pii/1385725872900340.

[8] Adrien Durier, Daniel Hirschkoff & Davide Sangiorgi (2022): *Eager functions as processes*. *Theoretical Computer Science* 913, pp. 8–42, doi:https://doi.org/10.1016/j.tcs.2022.01.043. Available at https://www.sciencedirect.com/science/article/pii/S0304397522000718.

[9] Daniele Gorla (2010): *Towards a unified approach to encodability and separation results for process calculi*. *Information and Computation* 208(9), pp. 1031–1053, doi:10.1016/j.ic.2010.05.002.

[10] Daniel Hirschkoff (1997): *A full formalisation of π-calculus theory in the calculus of constructions*. In Elsa L. Gunter & Amy Felty, editors: *Theorem Proving in Higher Order Logics*, Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 153–169, doi:https://doi.org/10.1007/BFb0028392.

[11] Daniel Hirschkoff (2003): *A brief survey of the theory of the Pi-calculus*. Research Report LIP RR-2003-13, Laboratoire de l'informatique du parallélisme. Available at https://hal-lara.archives-ouvertes.fr/hal-02101985.

[12] Paul Blain Levy (1999): *Call-by-Push-Value*: *A Subsuming Paradigm* (*extended abstract*), pp. 228–243. doi:10.1007/3-540-48959-2_17.

[13] Paul Blain Levy (2016): *A tutorial on call-by-push-value*. Available at https://www.cs.bham.ac.uk/~pbl/papers/cbpvefftt.pdf.

[14] Robin Milner (1990): *Functions as processes*. In Michael S. Paterson, editor: *Automata, Languages and Programming*, Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 167–180, doi:https://doi.org/10.1007/BFb0032030.

[15] Robin Milner, Joachim Parrow & David Walker (1992): *A calculus of mobile processes*, *I*. *Information and Computation* 100(1), pp. 1–40, doi:https://doi.org/10.1016/0890-5401(92)90008-4. Available at https://www.sciencedirect.com/science/article/pii/0890540192900084.

[16] G.D. Plotkin (1975): *Call-by-name*, *call-by-value and the λ-calculus*. *Theoretical Computer Science* 1(2), pp. 125–159, doi:https://doi.org/10.1016/0304-3975(75)90017-1. Available at https://www.sciencedirect.com/science/article/pii/0304397575900171.

[17] Christine Rizkallah, Dmitri Garbuzov & S. Zdancewic (2018): *A Formal Equational Theory for Call-By-Push-Value*, pp. 523–541. doi:10.1007/978-3-319-94821-8_31.

[18] Christine Rizkallah, Dmitri Garbuzov & Steve Zdancewic (2018): *A formal equational theory for call-by-push-value*. In: *International Conference on Interactive Theorem Proving*, Springer, pp. 523–541, doi:10.1007/978-3-319-94821-8_31.

[19] Davide Sangiorgi (1993): *An Investigation into Functions as Processes*. In: *Proceedings of the 9th International Conference on Mathematical Foundations of Programming Semantics*, Springer-Verlag, Berlin, Heidelberg, p. 143–159.

[20] Davide Sangiorgi (1995): *πI*: *A symmetric calculus based on internal mobility*. In: *Colloquium on Trees in Algebra and Programming*, Springer, pp. 172–186. Available at https://inria.hal.science/inria-00074139.

[21] Davide Sangiorgi (1996): *π-Calculus*, *internal mobility*, *and agent-passing calculi*. *Theoretical Computer Science* 167(1), pp. 235–274, doi:https://doi.org/10.1016/0304-3975(96)00075-8. Available at https://www.sciencedirect.com/science/article/pii/0304397596000758.

[22] Davide Sangiorgi (1996): *A theory of bisimulation for the π-calculus*. *Acta Informatica* 33(1), pp. 69–97, doi:10.1007/s002360050036. Available at https://doi.org/10.1007/s002360050036.

[23] Davide Sangiorgi (1999): *From λ to π*; *or*, *Rediscovering continuations*. *Mathematical Structures in Computer Science* 9(4), pp. 367–401.

[24] Davide Sangiorgi (1999): *The name discipline of uniform receptiveness*. *Theoretical Computer Science* 221(1), pp. 457–493, doi:10.1016/S0304-3975(99)00040-7. Available at https://www.sciencedirect.com/science/article/pii/S0304397599000407.

[25] Davide Sangiorgi (2019): *Asynchronous π-calculus at Work*: *The Call-by-Need Strategy*, pp. 33–49. Springer International Publishing, Cham, doi:10.1007/978-3-030-31175-9_3.

[26] Davide Sangiorgi & David Walker (2001): *PI-Calculus*: *A Theory of Mobile Processes*. Cambridge University Press, USA.

[27] Davide Sangiorgi & David Walker (2003): *The pi-calculus*: *a Theory of Mobile Processes*. Cambridge university press.